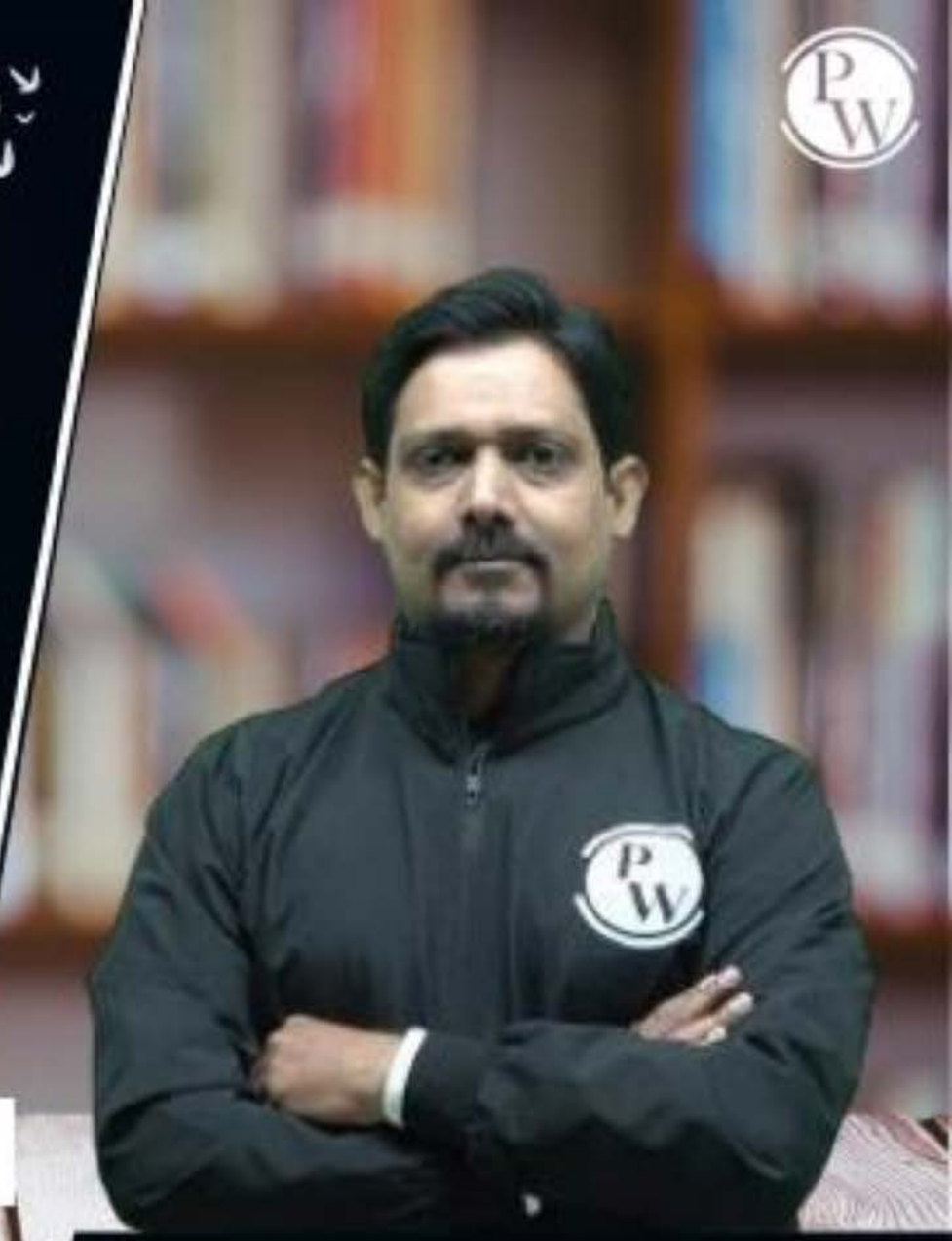
Computer Science & IT

Data Structure & Programming



Linked List

Lecture No. 02



By- Abhishek Sir

Recap of Previous Lecture



Topic — Basic of pointer

Topic — Basic of call by reference

Topic — Basic of structure

Topic — Basic of Dynamic Memory allocation

Topic

Slide

Topics to be Covered



- Topic
- Topic
- Topic
- Topic
- Topic

Linked List

Basic function Single List.

Slide



Topic Why Linked List

1. Size of array is fixed. : If you need more or less array is Not recommended.

2. Cost of Insert : ordered array — finding correct position & shifting



Array size dependent

Insert at the end

unordered array :



Inscrt at the end

Deleted shift

Cost deletion - : Ordered array

unordered array : We can copy Last element at deletedpos

Slide



Topic Why Linked List

Dynamic Memory allocation

Shifting : will discuss after we learn linked. (Insertion Sort Algorithm)

Slide

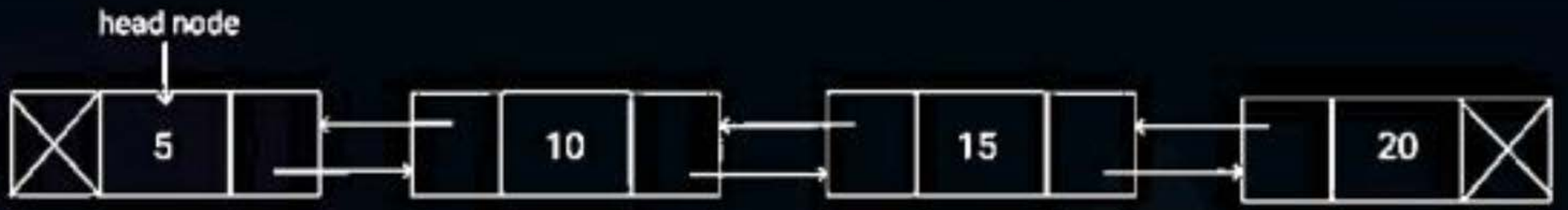


Topic : Linked List

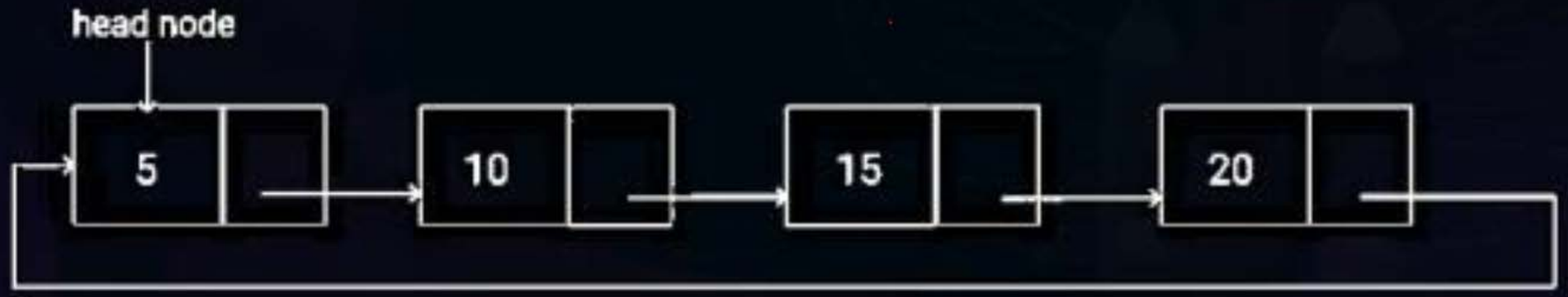
Singly Linked List



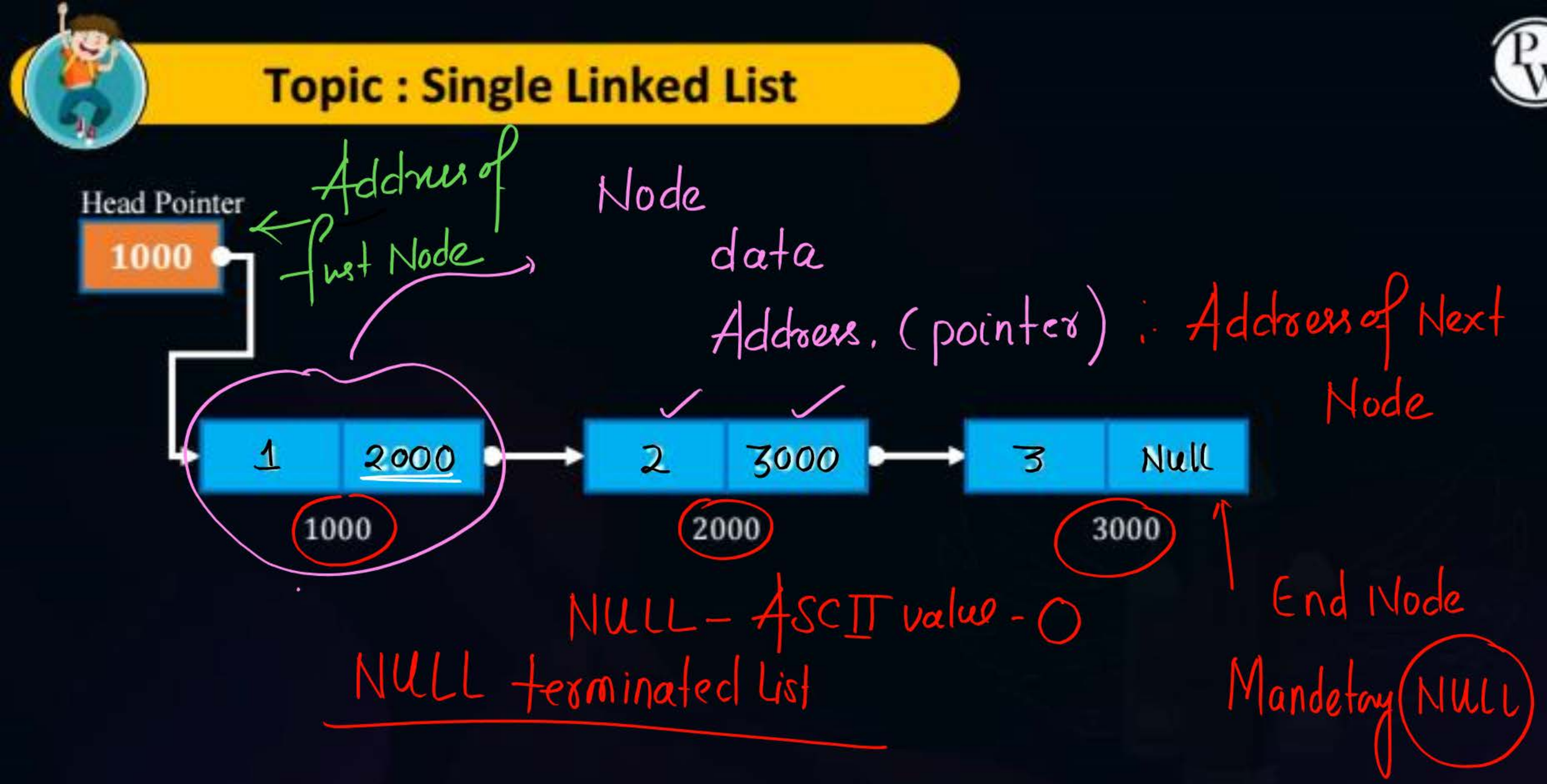
Doubly Linked List



Circular Linked List



Topic : Single Linked List



Slide



Topic : Single Linked List

* Head pointer contain Address of first Node of list.
* Last Node of list contains Next Address as NULL
* Each Node is having 2 part (1) data (2) Address of Next Node.

Slide



Topic : Single Linked List

```
typedef struct node{
        int data;
        struct node *next;
} Node;
```

→ Structure pointer

Self referential Address

node

data	Next

↑ alias

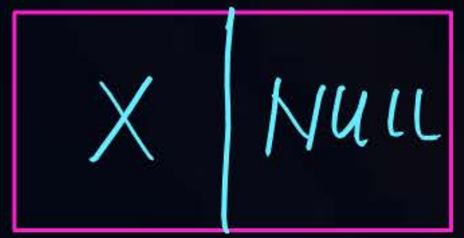
struct node or Node

Slide



Topic : Single Linked List

Node* getnode(int x) : getnode function creates a Linked list Node with data x and return address of it.



1000 (Assume)

return 1000

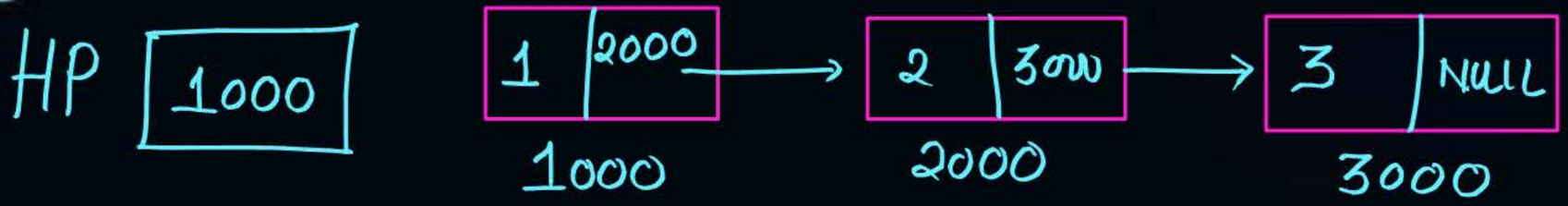
```
Node* getnode(int x){
    Node* p;
    p = malloc(sizeof(Node));
    p->data = x;
    p->next = NULL;
    return p;
}
```

Struct node

Slide



Topic : Single Linked List

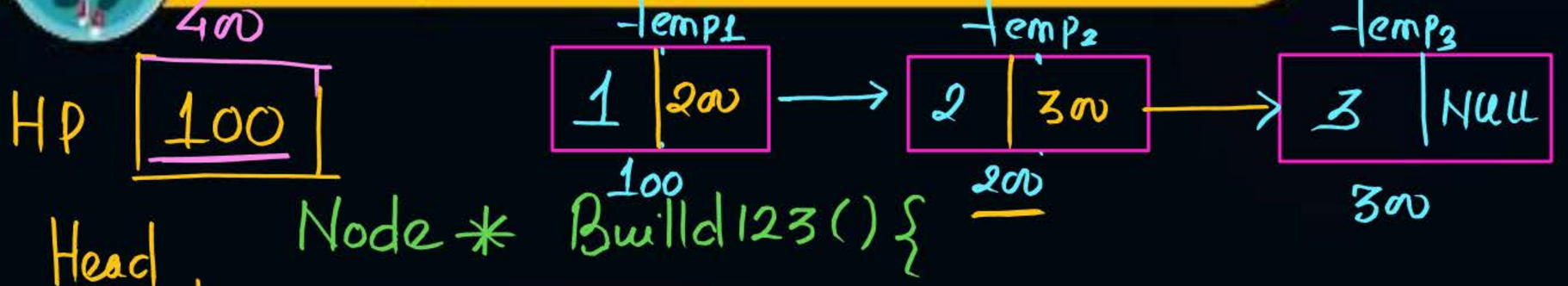


Node * Build123() A Linked List 1 → 2 → 3 will be created and address of first Node will be return.

Slide

Topic : Single Linked List





Node * Build123() {

Node *temp1, *temp2, *temp3;

temp1 = getnode(1);

temp2 = getnode(2);

temp3 = getnode(3);

temp1 → next = temp2;

temp2 → next = temp3;

return temp1;

}

pointer

temp1 100

1

temp2 200

temp3 300

2

Slide



Topic : Single Linked List

```
# include<stdlib.h>
# include<stdio.h>
int main(){

    Node * HP;

    HP = Build123();

    display(HP)

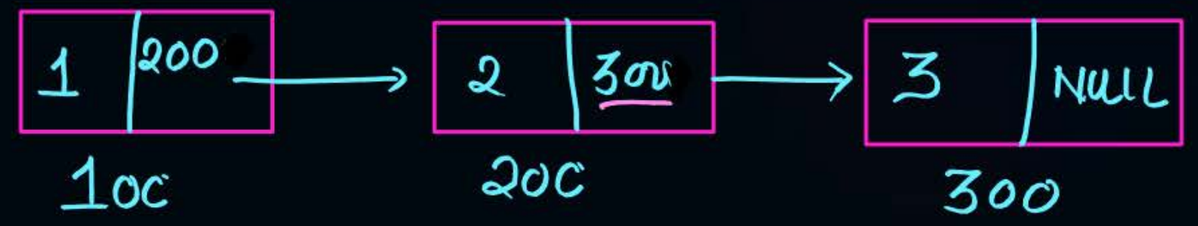
    printf("%d", count(HP));
```

```
void display(Node*)
int count(Node*)
```

Slide

Topic : Single Linked List





Nonempty void display (Node * S) { S 100

Traversal

print the list

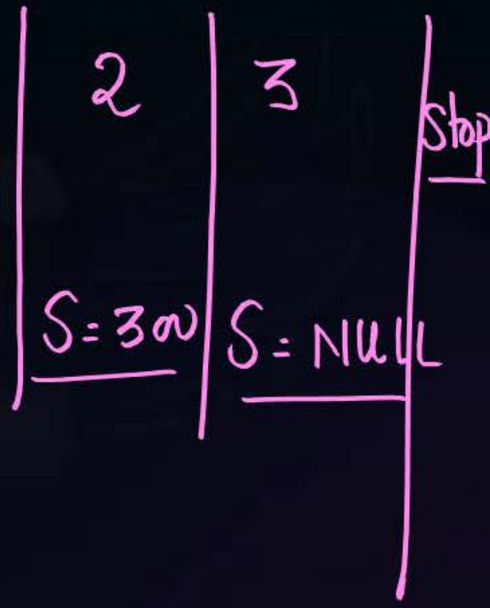
while (S != NULL) {

printf ("%d", S→data);

S = S→next;

}

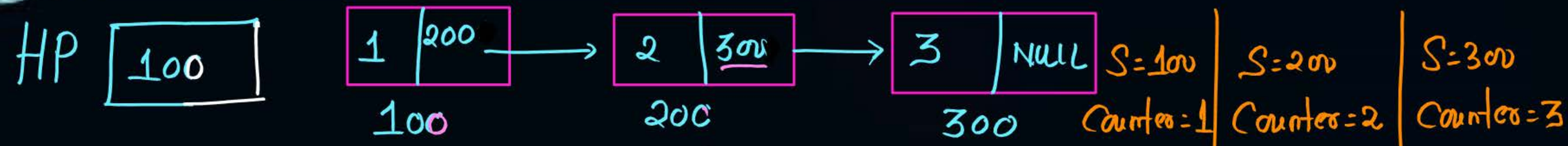
}



Slide

Topic : Single Linked List





No. of Node in list

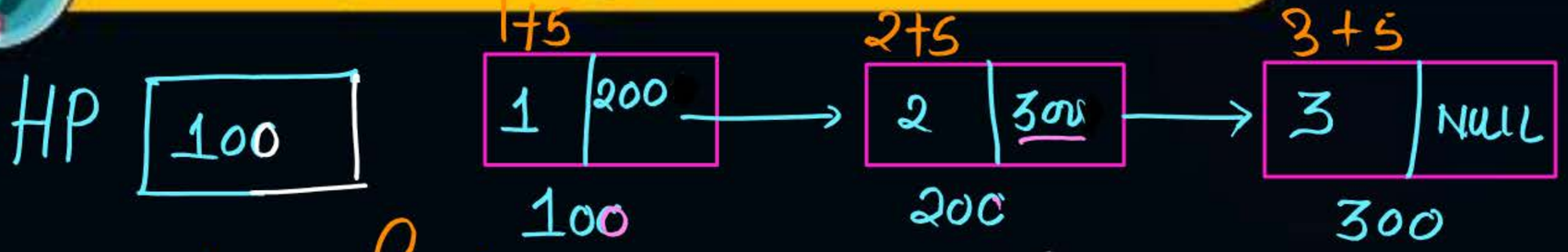
```
int count(Node *S){
    int counter=0;
    while(S!=NULL){
        counter=counter+1;
        S=S->next;
    }
    return counter;
}
```

return (3)

Slide

Topic : Single Linked List





Write a function that add 5 to every data value of Node

```
void fun(Node* HP){
  while(S!=NULL){
    S->data=S.data+5;// S->data+=5;
    S=S->next;
  }
}
```

S=100	S=200	300
S.data=6	S->data=7	S->data=8
S=200	S=300	S=NULL

Slide

Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

(A) A queue cannot be implemented using this stack.

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction

(GATE_2014_2 M)

Reverse

Enque = push

Dequeue - 2

Reverse pop

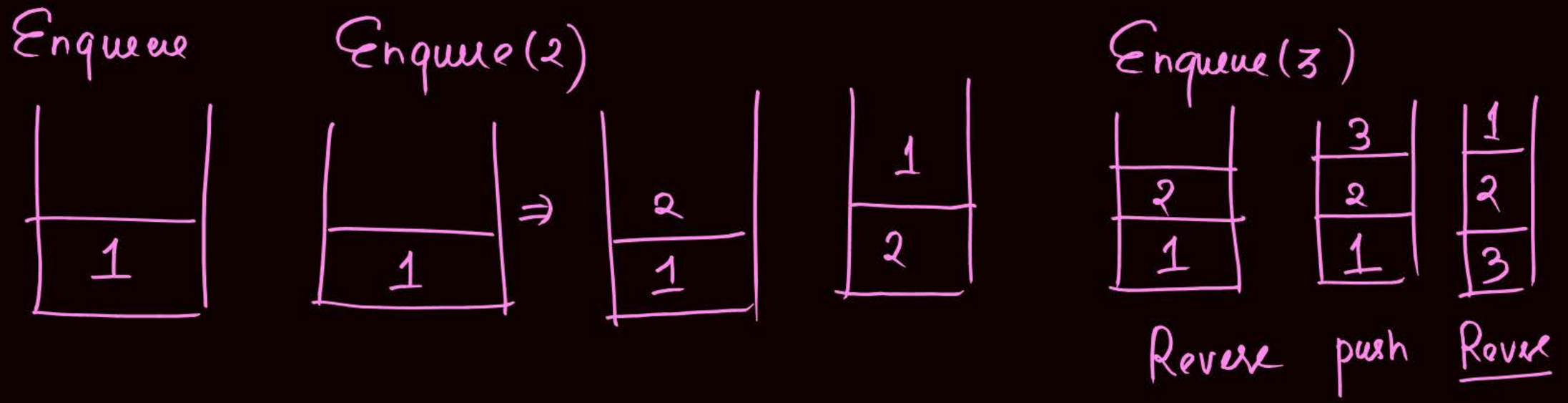
work only 1 time

Dequeue - pop

Reverse()

~~pop()~~ push

Reverse()


Enqueue
1
Enqueue(2)
1
⇒
2
1
1
2
Enqueue(3)
2
1
3
2
1
1
2
3
Reverse
push
Reverse



2 mins Summary

Topic — Single Linked List

Topic — getnode(int x)

Topic — Build123

Topic — display & count

Topic

Slide

THANK - YOU